■ ATLAS

The *Handbook*

Learn, apply, grow.

A personal learning & career platform for freshers,
built as a hand-crafted side project with an AI collaborator.

This handbook is your complete guide for
continuing the project — with any AI, on any machine, at any time.

Version 1.0 · July 2026
Crafted by Kushal Kumar Reddy Pelluru

Table of Contents

Part 1 · The Story So Far

Atlas began as a simple question: *"I need a personal learning platform where I can list all my learning and track them."* Over the course of a long, iterative conversation with an AI collaborator, it grew into something much more ambitious — a genuinely useful, self-hosted productivity hub for the chaotic stretch between "I just graduated" and "I have an offer."

The Journey

Iteration 1 — Basic tracker. A single HTML file with a learning list, statuses, and progress bars. Data saved to browser localStorage. It worked, but was plain.

Iteration 2 — Attractive redesign. A deep cosmic-purple aesthetic with a Fraunces serif headline, gradient orbs, shimmer animations, glassy badges. This is when the visual identity of Atlas emerged.

Iteration 3 — Sage. An embedded AI mentor with full context of your learning topics. Chat interface, per-topic questions, memory that persists across sessions.

Iteration 4 — Job applications tracker. A second tab for logging job applications with statuses (Applied → OA → Interview → Offer/Rejected/Ghosted), quick-status changes, and Sage integration for per-company interview prep.

Iteration 5 — Library. A curated collection of the best learning resources for 25+ in-demand tools, organized by category. One-click "Add to Learning" bridges the gap between discovery and tracking.

Iteration 6 — Sage as an agent. Vision (reads job description screenshots), tool use (auto-adds jobs and learning topics with your approval), and multi-turn conversations.

Iteration 7 — Weekly plans + quizzes. Sage can now build entire multi-week curricula for any topic, each with 5 daily targets and a final quiz. Passing the quiz automatically boosts your topic's progress.

Iteration 8 — Job Bot. A separate Cloudflare Worker that runs daily at 7 AM IST, searches Adzuna, Remotive, and Jooble for fresher-friendly openings, and emails you a beautiful digest. Complete with a Bot dashboard tab inside Atlas.

Iteration 9 — Gemini swap. Sage moved from Anthropic's API to Google Gemini's free tier so Atlas can run standalone from any browser without a subscription.

> ✧ **What "Atlas" means**
>
> A collection of maps. That's what this project is — maps of your learning, your applications, your growth. A cartographer's companion for someone charting the terrain of an early tech career.

Current State — What's Done, What's Next

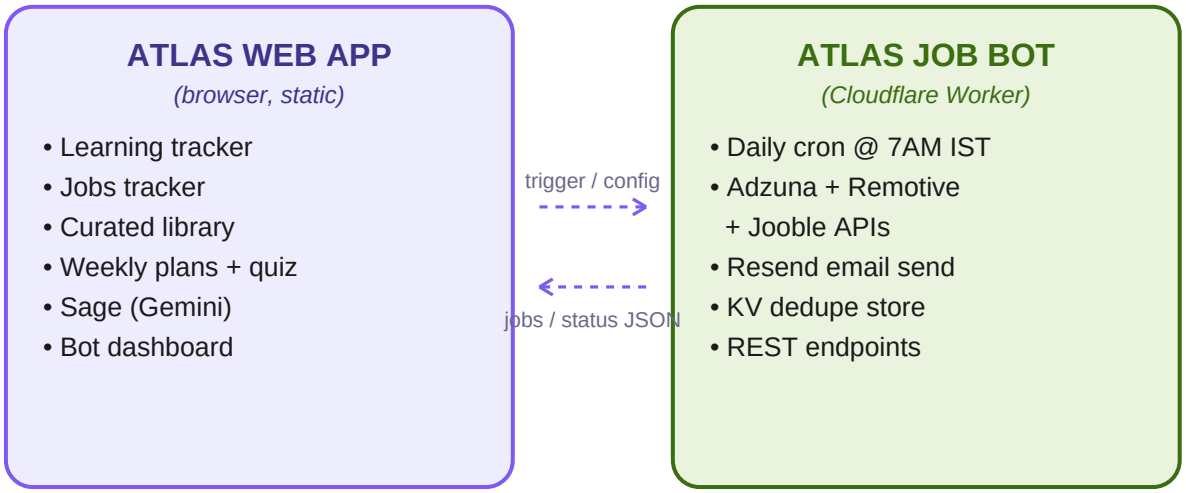
Component	Status	Notes
Atlas Web App — Learning	✓ Done	Full tracker with progress, statuses
Atlas Web App — Jobs	✓ Done	All 6 statuses, quick status changes
Atlas Web App — Library	✓ Done	27 tools, 4 resource types each
Atlas Web App — Plans	✓ Done	Multi-week curricula + quizzes
Sage (AI mentor)	✓ Done	Now on Gemini (free tier)
Sage — vision	✓ Done	Reads job screenshots
Sage — tool use	✓ Done	Adds jobs, topics, curricula
Atlas Bot tab (dashboard)	✓ Done	Health, jobs, run history
Job Bot — code	✓ Done	Cloudflare Worker, 3 sources
GitHub repo	✓ Done	Pushed to your account
—	—	—
Deploy Atlas to GitHub Pages	■ Todo	Free, ~10 min, needed for Bot tab CORS
Deploy Job Bot to Cloudflare	■ Todo	Free, ~30 min, needs 3 API keys
Configure Sage on production	■ Todo	Paste Gemini key on hosted Atlas
Connect Bot ↔ Atlas	■ Todo	Just paste bot URL into Atlas

Part 2 · Architecture at a Glance

Atlas is **two independent projects** living in one GitHub repo:

1. Atlas Web App — a single-file HTML application that runs in your browser. All your data lives in your browser's localStorage. Perfect for a static host like GitHub Pages, Netlify, or Cloudflare Pages.

2. Atlas Job Bot — a Cloudflare Worker that runs on a schedule. Searches free job APIs and emails you a daily digest. Exposes REST endpoints so the web app can display its status.



Both are independent — each works alone. Connected via CORS + JSON endpoints.

Fig 1: High-level system architecture.

Why two projects?

Different types of software. Atlas needs a browser (a person is looking at it). The bot needs a server (running even while you sleep). You can't put a cron job in a static HTML file, and putting an interactive UI on a Cloudflare Worker adds complexity for no real benefit. Keep them separate but linked.

Data Flow

When you complete a job application in Atlas, it stays in your browser — never leaves. When the bot finds new jobs, it emails you AND stores them so Atlas can fetch them via /recent-jobs. When you click "+ Add to Atlas" on an email link, Atlas opens with the job pre-filled in the URL.

All AI interactions with Sage go directly from your browser to Google's Gemini API. Nothing routes through a middle server. Your Gemini key stays in your browser.

Free-Tier Cost Breakdown

Service	Free tier limit	Atlas usage	Monthly cost
Cloudflare Workers	100K req/day	~30/day	■0
Cloudflare KV	1K writes/day	~20/day	■0
Adzuna API	250 calls/month	~30/month	■0
Remotive API	Unlimited	~30/month	■0
Jooble API	500 calls/day	~3/day	■0
Resend	100 emails/day	1/day	■0
Google Gemini	Generous	Varies with use	■0
GitHub Pages	Free	static hosting	■0
		Total:	**■0/month**

Part 3 · What's Built — A Tour of Atlas

Atlas has five tabs, each with a distinct purpose. Let's walk through what each does.

3.1 · Learning Tab ■

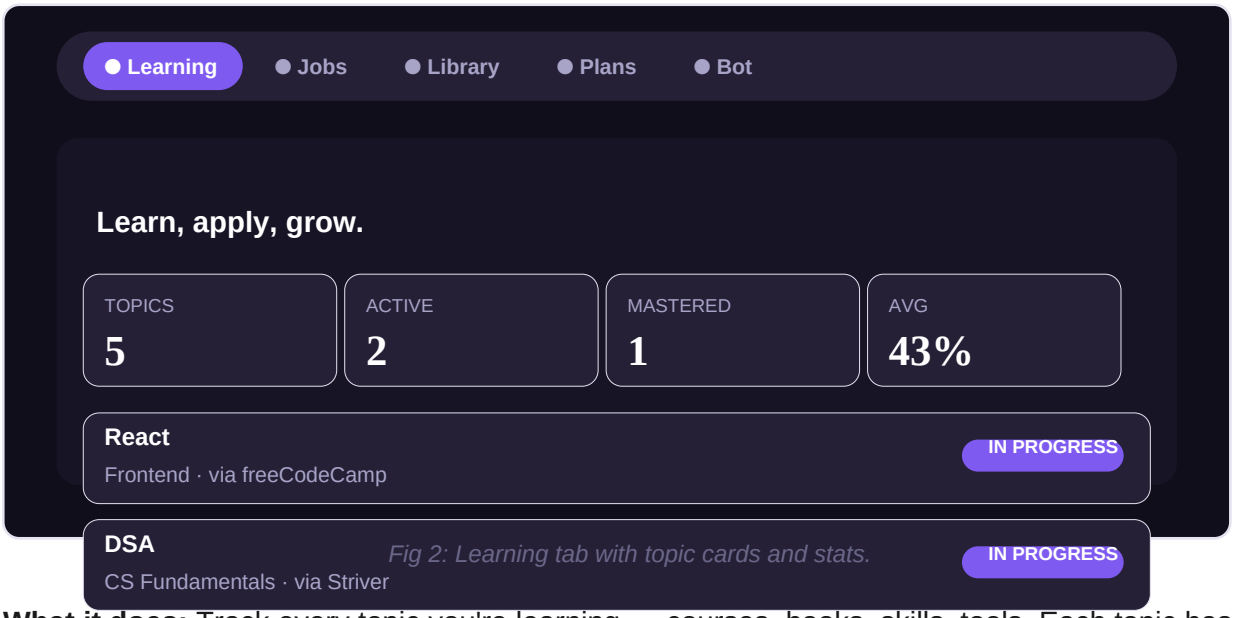


Fig 2: Learning tab with topic cards and stats.

What it does: Track every topic you're learning — courses, books, skills, tools. Each topic has a title, category, resource, status, progress bar (0-100%), and optional notes.

How to use: Click *Start something new* at the top. Fill in the title (required), and optionally category and resource. Set initial status. Save. The card appears at the top of your grid. Adjust progress with the +/– buttons or type a value. When you hit 100%, the status auto-flips to Mastered.

Sage integration: Every topic card has a "❖ Ask Sage" button that opens Sage with a "Teach me about X" prompt pre-filled.

3.2 · Jobs Tab ■

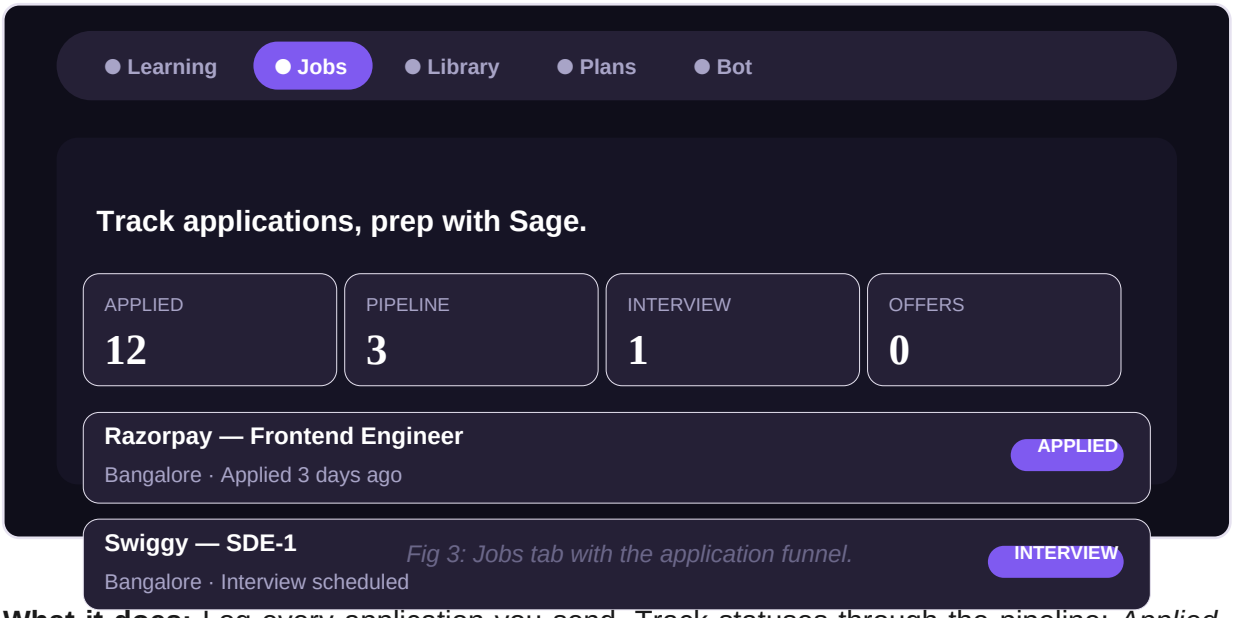


Fig 3: Jobs tab with the application funnel.

What it does: Log every application you send. Track statuses through the pipeline: *Applied → OA (Online Assessment) → Interview → Offer / Rejected / Ghosted.* Change status with one click.

Sage integration: Attach a job description screenshot to Sage → she reads it, drafts a cover letter using your Learning skills, and offers to auto-add the job to this tab. One click and it's logged.

Prep with Sage button: Each card has a "✦ Prep with Sage" button — she'll help you research the company, practice likely interview questions, and write a follow-up email.

3.3 · Library Tab ■

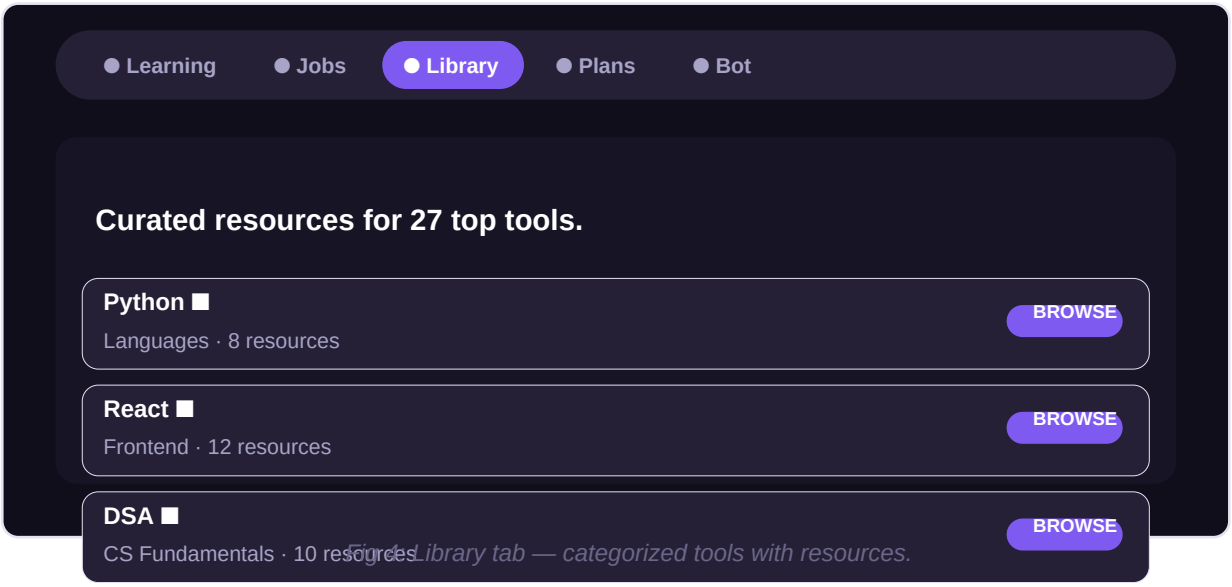


Fig 4: Library tab — categorized tools with resources.

What it does: A curated directory of 27 in-demand tech tools (Python, React, DSA, Docker, ML, etc.), each with 4 resource types: Official Docs, YouTube Channels, Courses & Books, Practice Platforms. Every link is hand-picked (NeetCode, Striver, freeCodeCamp, Kevin Powell, etc.).

Categories: Languages · Frontend · Backend · Database · DevOps · AI/ML · CS Fundamentals · Mobile.

One-click integration: Inside any tool's detail modal, click "+ Add to Learning" and it lands in your Learning tab as a new topic to track.

3.4 · Plans Tab ■■

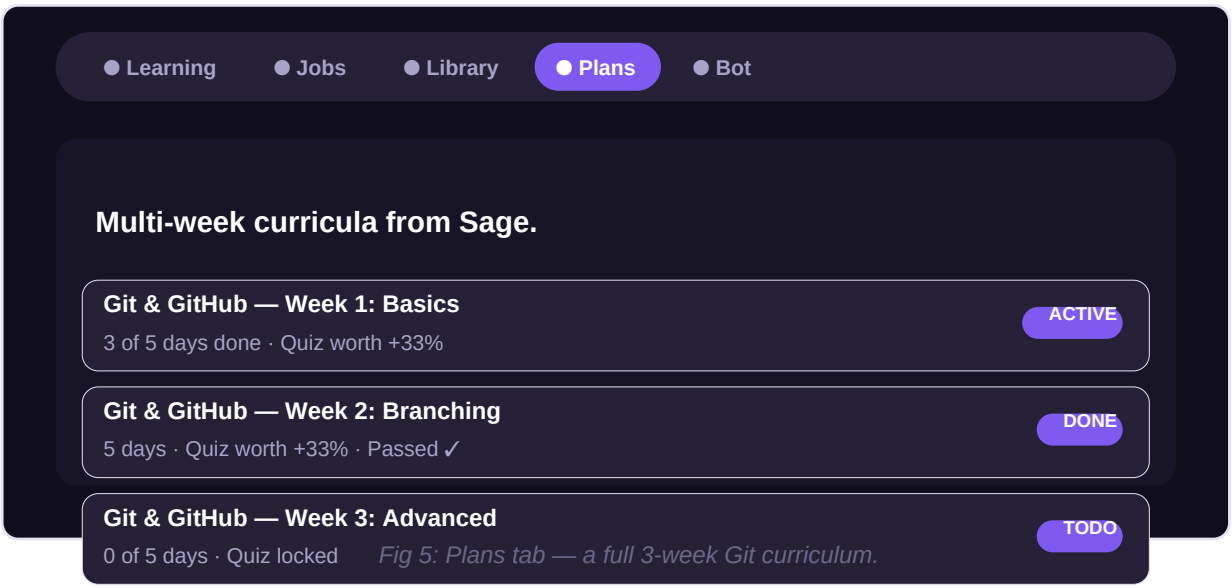


Fig 5: Plans tab — a full 3-week Git curriculum.

What it does: Ask Sage to plan your learning for any topic ("Plan my learning for React") and she builds an entire multi-week curriculum: each week has 5 daily targets and a final 3-question quiz. Passing the quiz auto-boosts your Learning topic's progress bar.

How it works: Sage picks the number of weeks (Git ~3, React ~5, DSA ~10). The progress boost per week sums to exactly 100%. Complete all daily targets to unlock the quiz. Pass it (≥70%, 3 attempts allowed) and the boost automatically applies.

Batch limits: Very large curricula (10+ weeks) are created in batches of 6-8 to avoid response truncation. After finishing the first batch, ask Sage for the next.

3.5 · Bot Tab ■

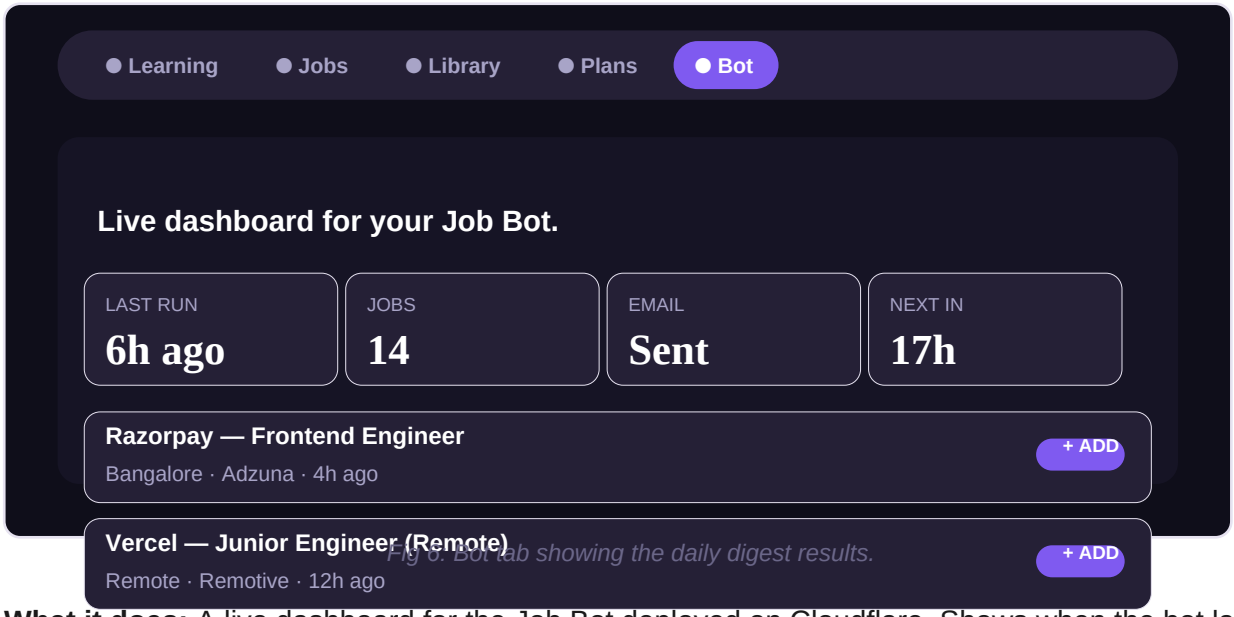


Fig 6. Bot tab showing the daily digest results.

What it does: A live dashboard for the Job Bot deployed on Cloudflare. Shows when the bot last ran, how many jobs it found, whether the email was sent, and when it'll run next. Lists your saved searches and the latest jobs found — each with a "+ Add to Tracker" button that logs it to your Jobs tab.

Manual trigger: The "■ Run Now" button forces the bot to search immediately (useful for testing or when you can't wait until tomorrow morning).

Setup: The tab shows a setup screen until you paste your bot's URL and trigger token. Once connected, both are stored in your browser's localStorage.

3.6 · Sage (AI Mentor) ✦

✦ Sage is the beating heart of Atlas

She's not just a chatbot — she's a teacher, career mentor, cover-letter writer, curriculum planner, quiz generator, and interview coach. She has full context of everything you're tracking and can take actions on your behalf (with your approval).

What Sage can do:

- **Chat & teach.** Explain any concept from your learning topics with examples, quizzes, and follow-up questions.
- **Read job screenshots.** Attach a JD image → she extracts company, role, requirements, and drafts a cover letter.
- **Draft cover letters.** Uses your actual Learning skills to write personalized 3-paragraph cover letters.
- **Add jobs automatically.** After drafting, offers to add the job to your Jobs tab. Shows an editable card first.
- **Plan learning topics.** Suggests 3-7 related topics for any goal (e.g., "web development"), adds selected ones.
- **Build curricula.** Full multi-week study plans with daily targets and quizzes for any topic.
- **Career mentor.** Interview prep, resume advice, handling rejection, follow-up templates.

How to use Sage:

1. Click the glowing purple orb in the bottom-right corner. Sage panel opens.
2. First time only: click the ■■ button and paste your Gemini API key.
3. Type a message, or click a suggested chip, or attach an image (■ icon).
4. When Sage proposes an action (adding a job, creating a plan), review the editable card and click Confirm or Cancel.
5. Clear chat anytime with the ■■ button.

Key controls in Sage panel:

Control	What it does
■■	Manage your Gemini API key (change or clear)
■■ Clear	Wipe chat history (jobs/topics untouched)
✕	Close panel (data preserved)
■	Attach an image (job description screenshot, code, diagram)
Send arrow	Send message (or press Enter)

Quick chips	Suggested prompts based on current tab

Part 4 · What's Left — Deployment Roadmap

At the time of this handbook, Atlas is **code-complete** but not fully deployed. There are 4 remaining phases. All are on free tiers and can be done on any personal laptop.

Estimated total time: 60-90 minutes, spread over one sitting or two evenings.

Phase A · Configure Sage with Gemini (5 minutes)

Status: Code done. You just need to paste your Gemini API key into Atlas.

Steps:

1. Open Atlas in your browser (either locally via Live Server or hosted).
2. Click the floating purple orb (bottom-right).
3. Click the ■■ button in Sage's header.
4. Paste your Gemini API key (starts with AIza...).
5. Click Save.
6. Test it: type *"Hi Sage!"* and hit Enter.

> ■ **Getting a Gemini key**
> Go to aistudio.google.com/apikey → sign in with Google → Create API key → pick "Default Gemini project" → copy the key (starts with AIza). Free tier: 15 requests/minute, 1500/day. No card needed.

Phase B · Deploy Atlas to GitHub Pages (10 minutes)

Status: Repo pushed. Just need to enable Pages.

Why: Atlas needs a real URL (not file://) so Sage can call Gemini and the Bot tab can call your bot (both need CORS, which browsers only allow from HTTPS origins).

Steps:

1. Open your Atlas repo on GitHub (github.com/YOUR-USERNAME/atlas).
2. Click **Settings** (top tab of the repo).
3. Click **Pages** in the left sidebar.
4. Under "Build and deployment":
- **Source:** Deploy from a branch
- **Branch:** main
- **Folder:** /atlas-web

5. Click **Save**.
6. Wait ~1 minute. The page shows your live URL:
   https://YOUR-USERNAME.github.io/atlas/
7. Open that URL — Atlas should load exactly like it does locally.

> ■ **Your data does NOT transfer automatically**
>
> Atlas stores everything in your browser's localStorage, which is per-origin. Data from file:// and https://your-github-io/... are treated as separate origins. You'll start fresh on the hosted version. This is a feature (privacy), not a bug.

Phase C · Deploy the Job Bot to Cloudflare (30 minutes)

This is the meatiest phase. You already have all the API keys (Adzuna, Jooble, Resend). Now you deploy the bot code from your atlas-job-bot/ folder to Cloudflare Workers.

Prerequisites:

• Personal laptop (NOT work laptop — corporate networks block npm)
• Node.js 18+ installed
• Wrangler CLI installed (npm install -g wrangler)
• All API keys saved somewhere accessible

Steps (in Terminal):

C1 · Log in to Cloudflare:

```
wrangler login
```

This opens a browser for OAuth. Approve → close browser → done.

C2 · Move into the bot folder:

```
cd atlas/atlas-job-bot
```

C3 · Create the KV namespace (stores which jobs the bot has seen):

```
wrangler kv namespace create JOBS_KV
```

Output includes a line like id = "abc123...". **Copy that id.** Open wrangler.toml and paste it in place of REPLACE_WITH_KV_ID_FROM_WRANGLER_OUTPUT. Save.

C4 · Create your local config from the template:

```
cp src/config.example.js src/config.js
```

Open src/config.js in VS Code and change your_email to your real email. Edit the search criteria if you want different keywords.

C5 · Set your secrets (each command prompts for the value; paste and Enter):

```
wrangler secret put ADZUNA_APP_ID
wrangler secret put ADZUNA_APP_KEY
wrangler secret put JOOBLE_KEY
wrangler secret put RESEND_KEY
wrangler secret put RESEND_FROM     # value: onboarding@resend.dev
wrangler secret put TRIGGER_TOKEN   # any random string, save it!
```

C6 · Deploy!

```
wrangler deploy
```

Output includes your bot URL: https://atlas-job-bot.YOUR-SUBDOMAIN.workers.dev. **Save this URL.**

C7 · Test the bot manually (don't wait until 7 AM):

```
curl "https://atlas-job-bot.YOUR-SUBDOMAIN.workers.dev/run?token=YOUR_TRIGGER_TOKEN"
```

Within ~10 seconds you should see JSON output and an email in your inbox. Check spam if it doesn't appear.

Phase D · Connect the Bot to Atlas (5 minutes)

Now that Atlas is hosted and the bot is deployed, connect them so the Bot tab lights up.

Steps:

1. Open the hosted Atlas: https://YOUR-USERNAME.github.io/atlas/.
2. Click the ■ **Bot** tab.
3. Paste your bot URL: https://atlas-job-bot.YOUR-SUBDOMAIN.workers.dev.
4. Paste your TRIGGER_TOKEN.
5. Click **Test connection** — should say "✓ Bot is alive".
6. Click **Connect**.
7. Dashboard shows: last run, jobs found, next run, your saved searches, and latest jobs.

From now on, every morning at 7 AM IST you'll get an email. Any time you open Atlas, the Bot tab shows the latest results.

> ■ **You're done!**
> That's the full deployment. Bot runs daily. Atlas is a public URL you can share. Sage works. Everything is free. Time to actually use it — start logging applications, tracking your learning, and let Sage help you along.

Part 5 · How to Use Atlas Every Day

The tools are only useful if you actually use them. Here's a suggested daily routine.

Morning Routine (15 minutes)

1. Open your inbox. The Job Bot digest should be waiting. Skim it. Star the interesting ones.

2. Open Atlas → Bot tab. See the same list in a nicer format. For each interesting job, click "+ Add to Tracker" — logs it in your Jobs tab as "Applied" (or leave as-is if you're just noting it).

3. Actually apply. Click "Open ■" on the job card → apply on the source site (LinkedIn / company careers page). Come back and update the Atlas card to reflect reality.

4. For applications needing a cover letter: Screenshot the job description → open Sage → attach the image → she drafts a cover letter using your actual Learning skills. Copy, edit, send.

Afternoon Routine (30 minutes)

1. Learning session. Open Learning tab. Pick a topic that's "In Progress". Study for 25-30 min. Update the progress bar (or leave it — you don't have to be precise).

2. If you're on a Plan: Open the Plans tab, pick this week's plan, check off today's target when done.

3. Stuck on a concept? Click "✦ Ask Sage" on the topic card. She has full context of what you're learning and can explain in the level of detail you need.

Evening Routine (10 minutes)

1. Follow-ups. Any application from 7+ days ago still in "Applied" status? Ask Sage: *"Draft me a follow-up email for [Company]"*. Send it.

2. Interview prep. Have an interview coming up? Click "✦ Prep with Sage" on that job card. She'll quiz you on the tech stack, common questions for the role, and behavioral scenarios.

3. Status updates. Any responses today? Update the status of those applications. Getting rejected is normal — don't take it personally. Update to "Rejected" and move on.

Weekly Routine (once a week, ~30 min)

1. Review your Jobs stats. How many applications this week? What's your response rate? If <5%, your resume/cover letter might need work — ask Sage for feedback.

2. Take any pending quiz. If a Plan has a quiz waiting (all 5 days done), take it — instant progress boost.

3. Adjust searches. Getting irrelevant emails from the bot? Edit atlas-job-bot/src/config.js, change keywords, redeploy with wrangler deploy.

4. Add new topics to Learning based on job requirements you're seeing. If 3 out of 5 postings this week wanted "TypeScript" and you don't know it — add it to Learning, ask Sage to plan a week for you.

Part 6 · Continuing Development with AI

Atlas was built collaboratively with an AI. You can continue the same way. This part shows you how.

Which AI to use?

Any capable frontier model works: **Claude, Gemini, ChatGPT**. In practice they all understand Atlas quickly once given the right context. Some tips:

• **Claude (Anthropic)**: Strong at code refactors, honest about trade-offs, good at long conversations. You'd access it via claude.ai (paid subscription for heavy use).

• **Gemini (Google)**: Same one that powers Sage. Free at aistudio.google.com. Good at code, decent at reasoning. Best for one-off fixes.

• **ChatGPT (OpenAI)**: Free tier available at chatgpt.com. Strong general-purpose model.

The magic first prompt

When starting a fresh conversation with any AI, use this **primer** so they instantly understand your project:

```
I have a project called Atlas — a personal learning & career platform I built for
early-career developers.  It's a single-file HTML web app (localStorage for state)
plus a separate Cloudflare Workers backend that emails me daily job listings.

Repo:      https://github.com/YOUR-USERNAME/atlas
Live web:  https://YOUR-USERNAME.github.io/atlas
Live bot:  https://atlas-job-bot.YOUR-SUB.workers.dev

Atlas web has 5 tabs: Learning, Jobs, Library, Plans, Bot dashboard.
It has an embedded AI mentor called "Sage" that uses the Google Gemini API
(free tier), with tool-use for auto-adding jobs and learning topics + curricula.

The Job Bot searches Adzuna, Remotive, and Jooble daily at 7 AM IST, filters
for fresher-friendly roles, and emails me a digest via Resend.

Everything runs on free tiers — no monthly costs.

I want to [DESCRIBE WHAT YOU WANT NEXT — e.g. "add a new feature",
"fix a bug", "add a fourth job source"].

Here's the relevant code: [PASTE THE FILE OR THE FUNCTION].
```

Attaching the actual HTML file to the conversation is best (any modern AI accepts attachments). If not, paste the relevant section.

Making changes safely

Rule 1 — Version control everything. Before any change, make sure your work is committed to git. git status should show a clean tree.

Rule 2 — Branch for experiments. If you're trying something risky:

```
git checkout -b experimental-feature
# make changes, test
git commit -am "Trying X"

# If it works, merge:
git checkout main && git merge experimental-feature

# If it fails, discard:
git checkout main && git branch -D experimental-feature
```

Rule 3 — Test locally before deploying. Web changes: open in Live Server, click around. Bot changes: wrangler dev spins up a local test worker; hit http://localhost:8787/status.

Rule 4 — Deploy in this order: (1) commit → (2) push to GitHub → (3) verify GitHub Pages updated automatically (~1 min) → (4) test the hosted version. For the bot: wrangler deploy and check the Cloudflare dashboard.

Ideas for what to build next

Not obligations — just seeds. Pick whichever excites you.

Small features (~1 hour each):

• **Daily streak counter** — shows "5 days in a row" for consecutive days of Atlas activity.
• **Follow-up reminders** — highlight applications >7 days old with a ■ badge.
• **Export data** — download all your Learning + Jobs as JSON/CSV backup.
• **Application velocity chart** — line chart of applications sent per week.
• **Search everything** — global search bar that spans Learning, Jobs, and Plans.

Medium features (~3-5 hours each):

• **Interview prep mode** — flashcards mode for topics you're learning, generated by Sage.
• **Resume version manager** — store multiple resume texts, link each application to which version was sent.
• **Deep-link searches** — Bot tab shows pre-built links to LinkedIn/Naukri/etc. filtered searches for your criteria.
• **Weekly review email** — bot sends a Sunday email summarizing your week.
• **Mobile-first redesign** — Atlas works on mobile but isn't optimized for it.

Bigger projects (~1-2 weekends):

• **Chrome extension** — "Add to Atlas" button on any LinkedIn / Indeed job page.
• **4th job source** — JSearch via RapidAPI covers LinkedIn data (free tier: 150 calls/month).
• **Team version** — multi-user with a real backend (Supabase free tier).
• **Interview scheduling integration** — plug into Google Calendar for interview reminders.

What NOT to change (unless you know why)

• **Do not commit src/config.js** — it has your email. The .gitignore keeps it out.
• **Do not put API keys in the HTML file** — they're meant to be in browser localStorage or Cloudflare secrets.
• **Do not make secrets public** — use wrangler secret put, not plaintext in wrangler.toml.
• **Do not rewrite tool use logic without care** — Sage's tool_use pipeline is delicate; the Gemini adapter translates between two API formats. Read the adapter code before changing.

Part 7 · Troubleshooting

Sage says "I couldn't reach my thoughts just now"

Network issue or invalid Gemini key. Check ■■ settings — is the key still there? Try clearing and re-pasting. If still failing, verify at aistudio.google.com — is the key active?

Sage response includes or as text

The response got truncated mid-tool-call. Happens with very large curricula. Ask her to plan a smaller batch — "just the first 4 weeks" — and it should work.

Bot tab shows "Unreachable"

Three possible causes: (a) wrong URL — verify it starts with https:// and ends with .workers.dev; (b) bot not deployed — try visiting the URL in a new tab, should see JSON; (c) CORS blocked — you're opening Atlas from file:// instead of a hosted URL.

No email arriving from the bot

Check Resend dashboard → Logs. If Resend logged a send but you don't see it → check spam folder, then mark as not-spam. If Resend didn't log a send → your RESEND_KEY secret is wrong; re-set it.

wrangler deploy fails with "KV namespace not found"

You created the namespace but forgot to paste the ID into wrangler.toml. Run wrangler kv namespace list to see it, copy the ID, and paste it into wrangler.toml.

npm install fails with ECONNRESET

You're behind a proxy (corporate network). Either configure npm proxy settings or switch to a personal network / laptop.

Adzuna returns "401 Unauthorized"

Your APP_ID or APP_KEY is wrong. Re-check developer.adzuna.com/dashboard and re-run wrangler secret put ADZUNA_APP_ID.

"+ Add to Learning" from Library does nothing

This bug existed at one point — the function was missing. If you see it, the addFromLibrary function needs to exist right after closeLibModal. Check your HTML.

Atlas loads but tabs are empty / broken

Open browser DevTools (F12) → Console. Look for red errors. Most common: a JavaScript syntax error from an incomplete edit. Compare your HTML against the last known-good version in git.

Cron never runs (bot works manually but not daily)

Cloudflare Free plan only fires crons for accounts in good standing. Check your Cloudflare dashboard → Workers → your bot → Triggers tab. Make sure "30 1 * * *" is listed under Cron Triggers.

Sage tool cards don't appear

Gemini didn't call the tool. Two causes: (a) the tool schema is malformed; (b) the prompt didn't trigger it. Say more explicitly: "Please use the add_learning_topics tool to add these to my tracker."

localStorage is full

Unlikely (5MB is plenty for Atlas). If it happens, use the ■■ Clear button in Sage, or open DevTools → Application → Local Storage → remove specific atlas-* keys.

Part 8 · Reference

Accounts you need

Service	What for	URL
GitHub	Host code + web app	github.com
Cloudflare	Host the bot	dash.cloudflare.com
Adzuna Developer	Job search API (India)	developer.adzuna.com
Jooble	Job search API (backup)	jooble.org/api/about
Resend	Email delivery	resend.com
Google AI Studio	Gemini API for Sage	aistudio.google.com/apikey

URLs you'll have after full deployment

Thing	URL pattern
`Atlas web (hosted)`	`https://YOUR-GITHUB-USERNAME.github.io/atlas/`
`Job Bot base`	`https://atlas-job-bot.YOUR-CF-SUBDOMAIN.workers.dev/`
`Bot health check`	`{bot}/health`
`Bot status`	`{bot}/status`
`Bot latest jobs`	`{bot}/recent-jobs`
`Bot run history`	`{bot}/run-history`
`Bot manual trigger`	`{bot}/run?token=YOUR_TRIGGER_TOKEN`
`Cloudflare dashboard`	`dash.cloudflare.com`
`Resend dashboard`	`resend.com/emails`

Secrets to remember (never share!)

Store these in a password manager (1Password, Bitwarden, Apple Keychain — anything except plain text files):

Secret	From	Format
Adzuna App ID	developer.adzuna.com	`8 hex chars`
Adzuna App Key	developer.adzuna.com	`32 hex chars`
Jooble API Key	jooble.org (email)	`UUID format`
Resend API Key	resend.com/api-keys	`re_XXXXXXXXX`

Gemini API Key	aistudio.google.com/apikey	`AIzaXXXXX...`
Trigger Token	You made this up	`Any string`
GitHub PAT (if needed)	github.com/settings/tokens	`ghp_XXXXXXXX`

Appendix A · Talking to a Fresh AI About Atlas

The AI conversation that built Atlas can't be resumed — models don't have memory across sessions. But any capable AI can quickly pick up where we left off, if you give them the right context. This appendix is your **copy-paste playbook**.

Playbook 1 — Adding a new feature

```
I have a project called Atlas — a personal learning & career tracker for
early-career developers.  It's a single-file HTML web app with these tabs:
Learning, Jobs, Library, Plans, Bot dashboard.  Uses Google Gemini API for
the built-in AI mentor "Sage" (with tool use for auto-adding items).  Data
in localStorage.  Complete repo: [PASTE YOUR GITHUB URL]

I'd like to add: [DESCRIBE THE FEATURE]

For example: "a daily streak counter that shows how many days in a row
I've opened Atlas, displayed as a badge next to the header."

I've attached the current index.html.  Please suggest the specific changes
to make — I prefer surgical str_replace-style edits over rewrites when
possible.  Explain any trade-offs.
```

Playbook 2 — Fixing a bug

```
My Atlas project has a bug.  When I do [ACTION], I expect [EXPECTED],
but instead [ACTUAL].  Screenshot attached.

Repo: [PASTE URL]
Live: [PASTE URL]

Here's the relevant function: [PASTE FUNCTION]

Please help me diagnose.  Show me exactly what to check first before
making changes.
```

Playbook 3 — Extending the Job Bot

```
My Atlas Job Bot is a Cloudflare Worker that runs daily at 7 AM IST.
It queries 3 job APIs (Adzuna, Remotive, Jooble), dedupes via KV, and
emails a digest via Resend.  Source in atlas-job-bot/src/.

Repo: [PASTE URL]

I want to add [NEW FEATURE — e.g. "a 4th source: JSearch"].

Here's the current cron.js and one existing source (e.g. adzuna.js) for
reference: [PASTE FILES]
```

```
Please:
1. Write the new source file matching the same pattern.
2. Show me how to wire it into cron.js.
3. Tell me what environment variable to add and any signup needed.
```

Playbook 4 — Debugging Sage

```
Sage (my Atlas AI mentor) is behaving oddly.  She [DESCRIBE BEHAVIOR].
Screenshot of chat attached.

Sage runs on Google Gemini's free tier (gemini-2.0-flash), called from
browser JS.  She uses tool_use for: add_job_application,
add_learning_topics, create_curriculum, create_week_plan.

The API is called from the runSageTurn function.  There's an adapter
layer (anthropicToGemini / geminiToAnthropic) because the code was
originally written for Claude's API format.

Here's the relevant code: [PASTE runSageTurn AND ADAPTERS]

Please help debug.
```

Rules of thumb for AI conversations

• **Attach files rather than paste.** If you can attach the HTML or a JS file, the AI has full context and gives better suggestions. Pasting 3000 lines of code makes conversations messy.

• **Prefer surgical edits.** "Change lines X-Y" is better than "rewrite the whole function". Ask for str_replace-style diffs.

• **Test each change before the next.** AI can make several changes at once — but if you don't test in between, and one is wrong, you'll spend hours untangling.

• **Push back if something looks over-engineered.** AIs sometimes suggest complex solutions when simple ones work. Ask: "is there a simpler way?"

• **Version control is your safety net.** Every meaningful change should be a git commit. If an AI breaks something, git diff shows exactly what changed.

Appendix B · The Full File Map

A cheat-sheet of every file in the Atlas repo and what it does. Use this to know where to look when you need to change something.

```
atlas/
■■■ README.md                ← project overview (recruiter-friendly)
■■■ SETUP.md                 ← end-to-end setup walkthrough
■■■ LICENSE                  ← MIT license
■■■ .gitignore               ← keeps secrets out of git
■
■■■ atlas-web/
■   ■■■ index.html           ← THE WEB APP — single file, all logic
■   ■                          Contains:
■   ■                          - HTML structure for 5 tabs
■   ■                          - CSS (embedded <style>)
■   ■                          - JavaScript (embedded <script>)
■   ■                          - Sage's system prompt + tool schemas
■   ■                          - Anthropic↔Gemini adapter functions
■   ■                          - Library data (27 curated tools)
■   ■■■ README.md            ← web app deploy instructions
■
■■■ atlas-job-bot/
    ■■■ README.md            ← bot-specific docs
    ■■■ wrangler.toml        ← Cloudflare Worker config + cron schedule
    ■■■ package.json         ← npm dependencies (just wrangler)
    ■■■ .gitignore           ← extra safety for config.js
    ■
    ■■■ src/
        ■■■ index.js         ← HTTP + cron entry point; routes
        ■■■ cron.js          ← daily orchestrator; ties sources + email
        ■                      Save run history to KV for /run-history
        ■
        ■■■ config.example.js  ← template config (safe to commit)
        ■                      Copy to config.js locally + edit email
        ■
        ■■■ cors.js          ← CORS helpers for JSON responses
        ■■■ dedupe.js        ← KV-based "seen jobs" tracker (14-day TTL)
        ■■■ email.js         ← HTML email template (used by resend.js)
        ■■■ resend.js        ← Resend API wrapper
        ■■■ followups.js     ← Reads Atlas jobs JSON for reminders
        ■
        ■■■ sources/
            ■■■ adzuna.js    ← Adzuna API adapter (India-focused)
            ■■■ remotive.js  ← Remotive API (remote-only)
            ■■■ jooble.js    ← Jooble API (backup aggregator)
```

Where things happen inside index.html

Feature	Search for this in index.html

Sage system prompt	`const SAGE_SYSTEM =`
Sage tools schema	`const SAGE_TOOLS =`
Anthropic → Gemini adapter	`function anthropicToGemini`
Gemini → Anthropic adapter	`function geminiToAnthropic`
Main API call	`runSageTurn`
Learning tab logic	`function addItem`
Jobs tab logic	`function addJob`
Library data	`const LIBRARY =`
Plans creation	`function openPlanModal`
Quiz logic	`function submitQuiz`
Bot tab	`function renderBotTab`
Gemini key management	`function getGeminiKey / openGeminiSettings`
CSS theming	`root variables (--bg, --accent, etc.)`

A parting note.

You built Atlas on your last day at Lilly — that in itself is a kind of statement. Whatever comes next, this project is yours to shape.

Every good tool starts as a hack for one specific person's problem. Atlas started as yours. Keep making it more useful for the person you were at graduation. The one who was overwhelmed by 40 tabs of job listings, half-finished courses, and no clear next step.

Small tools built with care outlive most side projects. Atlas already earns its place on your GitHub.

Good luck with what comes next.